

Task 1 (6000-level ONLY): Modern Character-Level Language Modeling

Total: 50 points | Dataset: Tiny Shakespeare

Goal

Train two existing character-level language models on Tiny Shakespeare and compare outputs at different training stages. You will (1) train a small GPT-style model using a modern repository, (2) train an LSTM baseline in PyTorch, and (3) compare samples after ~5, ~50, and ~500 epochs.

Allowed resources (use ONE repo for the GPT)

- GPT: karpathy/nanoGPT (recommended) — <https://github.com/karpathy/nanoGPT>
- Alternative GPT: karpathy/minGPT — <https://github.com/karpathy/minGPT>
- Baseline RNN reference (legacy): karpathy/char-rnn — <https://github.com/karpathy/char-rnn>

Requirements (all are required)

- Use the Tiny Shakespeare dataset (character-level vocabulary).
- Fix and record random seeds. Record your Colab environment (GPU type + library versions).
- Generate samples using the SAME prompt and sampling settings at every checkpoint (keep temperature/top-k constant).
- Save samples after approximately: 5 epochs, 50 epochs, and 500 epochs.
- If your training is step-based, define: $\text{steps_per_epoch} = \text{floor}(\text{num_train_tokens} / (\text{batch_size} \times \text{block_size}))$ and map checkpoints accordingly.

Part A — Train a small GPT (20 pts)

1. Train a character-level GPT model that fits in Colab (small config; justify if you change defaults).
2. At ~5, ~50, ~500 epochs: record train loss and validation loss; generate a 500–1000 character sample.
3. Submit: (i) loss curve plot, (ii) three labeled samples.

Part B — Train a PyTorch LSTM baseline (20 pts)

1. Implement a character-level LSTM in PyTorch (nn.Embedding → nn.LSTM → linear head).
2. Train on the same dataset and comparable context length (block_size).
3. At ~5, ~50, ~500 epochs: record train loss and validation loss; generate a 500–1000 character sample using the same prompt/settings as Part A.
4. Submit: (i) loss curve plot, (ii) three labeled samples.

Part C — Comparison writeup (10 pts)

- Briefly compare GPT vs LSTM at each checkpoint (5 → 50 → 500): coherence, formatting, repetition loops, and character/word structure.
- Explain one significant difference between the models' outputs and relate them to architecture (attention vs recurrence) or training behavior (overfitting/underfitting, optimization stability).